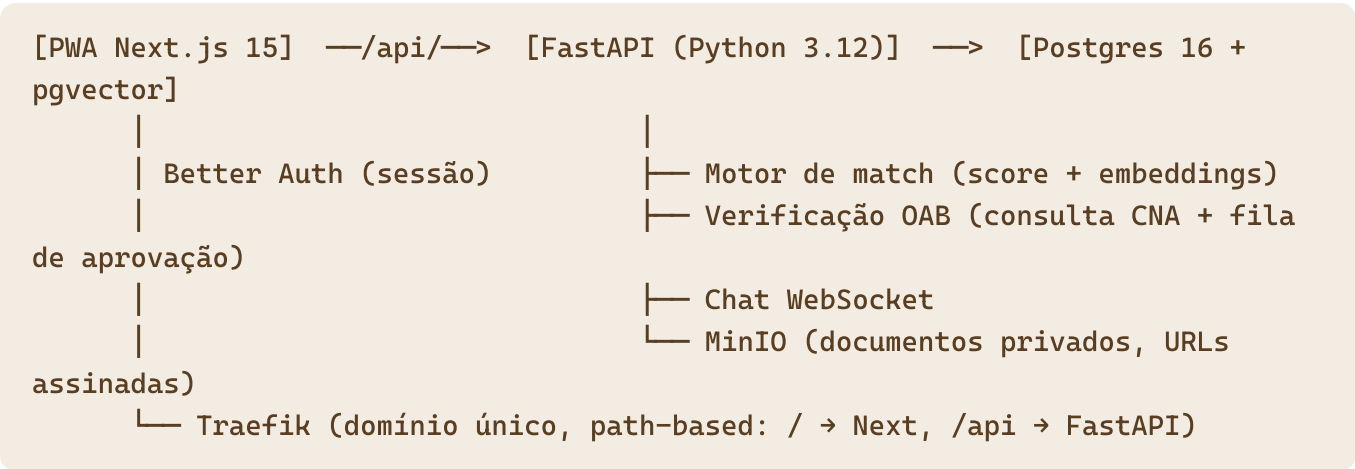


Stack e Arquitetura — Nextrigon

Premissas: time já domina **Next.js + Postgres**; cliente quer possibilidade de **Python no backend para maior proteção de dados**; MVP em 15 dias; deploy no padrão Raya self-hosted (VPS + Easypanel + Traefik).

Opção A — Next.js + FastAPI (recomendada)

A mesma arquitetura do **Mira** (mira.rayastudio.com.br), que já roda em produção: Next.js no front + FastAPI atrás do Traefik com roteamento por path. Zero curva de aprendizado de infra.



Camada	Tecnologia	Por quê
Frontend	Next.js 15 (App Router) + Tailwind + PWA (manifest + service worker)	Familiar; protótipo é mobile-first; instalável sem loja
Auth	Better Auth (no Next) com sessão compartilhada via JWT pro FastAPI	Padrão Raya; FastAPI valida o token, não duplica auth
API de dados	FastAPI + SQLAlchemy + Alembic	Python onde importa: dados sensíveis, match, verificação
Banco	Postgres 16 + pgvector	Embeddings de especialidade no próprio banco, sem serviço extra
Realtime (chat)	WebSocket nativo do FastAPI (+ fallback polling)	Sem dependência externa; Postgres LISTEN/NOTIFY pra fan-out
Arquivos	MinIO (bucket privado, presigned URLs com expiração)	Carteira OAB/selfie nunca ficam públicas

Camada	Tecnologia	Por quê
Embeddings	API externa (OpenAI text-embedding-3-small ou Voyage) com cache no banco	Centavos de custo; só recalcula quando o perfil muda
Infra	VPS Hostinger + Easypanel + Traefik; staging + prod	Padrão Raya, replicável do Mira

Por que Python = "maior proteção de dados" (argumento pro cliente):

1. O front **nunca** toca o banco — toda leitura/escrita passa pela API com validação (Pydantic), RBAC e rate limiting.
2. Documentos sensíveis (carteira, selfie) só existem no MinIO privado; a API emite URLs assinadas com expiração curta e **loga cada acesso** (trilha de auditoria LGPD).
3. Superfície de ataque menor: o container do Next não tem credencial de banco nem de storage.
4. Ecossistema Python para a evolução de IA (fase 2: re-rank LLM, aprendizado com swipes) — o motor de match já nasce na linguagem certa.

Custo da opção: dois serviços para manter (deploy, logs, contrato de API entre eles).

Mitigação: OpenAPI gerado pelo FastAPI + cliente TypeScript gerado automaticamente.

Opção B — Next.js full-stack (fallback de velocidade)

Next.js único com API Routes + Drizzle ORM direto no Postgres (padrão Raya Portal).

Python entra só como **worker** do motor de match (job que recalcula scores e embeddings, sem API exposta).

-  Mais rápido de entregar (um serviço, um deploy, um repo)
-  Time já fez exatamente isso (Raya Portal, Lyrallite)
-  Sem a separação de dados que o cliente pediu — credenciais de banco no mesmo processo que serve o front
-  WebSocket em Next self-hosted é mais chato (precisa de custom server ou polling)
-  A história de "proteção de dados com Python" vira só meio verdadeira

Quando escolher B: se no D5 o cronograma estiver derrapando, a Opção B é o plano de contingência — o schema do banco é o mesmo, então a migração de A→B (ou B→A depois) não joga trabalho fora.

Recomendação

Opção A. O pedido explícito do cliente (Python para proteção de dados) + precedente do Mira em produção + o motor de match/IA ser o coração do produto fazem o custo dos dois serviços valer a pena. Better Auth emite JWT; FastAPI valida — integração já resolvida no Mira.

Decisões fechadas

- ☒ ~~PWA mobile-first (não nativo) — 2026-06-10~~
- ☒ ~~Pagamentos fora do MVP — 2026-06-10~~
- ☐ Opção A vs B — **validar com Brendow no D0**
- ☐ Domínio de staging (sugestão: nextrigon-staging em subdomínio Raya até o cliente apontar o domínio dele)
- ☐ Provider de embeddings (OpenAI vs Voyage) — decisão de D5, irrelevante pro schema

Relacionados: [Nextrigon](#) · [02-PRD](#) · [04-Cronograma-15-Dias](#)